

WEEK-2

Aggregate Functions

J. Simran

2520030166

Step 1: Create Database

The screenshot shows the SQL Developer interface with the following SQL script in the main editor:

```
1 • CREATE DATABASE IF NOT EXISTS bank_db;
2 • USE bank_db;
```

The Output window displays the execution results:

#	Time	Action	Message
1	14:57:53	CREATE DATABASE IF NOT EXISTS bank_db	1 row(s) affected, 1 warning(s): 1007 Can't create database '...
2	14:57:53	USE bank_db	0 row(s) affected

Step 2: Create Table

The screenshot shows the SQL Developer interface with the following SQL script in the main editor:

```
1 • CREATE DATABASE IF NOT EXISTS bank_db;
2 • USE bank_db;
3 • DROP TABLE IF EXISTS bank_transactions;
4 • CREATE TABLE bank_transactions (
5     txn_id INT PRIMARY KEY,
6     customer_name VARCHAR(50),
7     branch_name VARCHAR(50),
8     transaction_type VARCHAR(20),
9     amount DECIMAL(10,2),
10    transaction_date DATE
11 );
```

The Output window displays the execution results:

#	Time	Action	Message
69	15:07:11	SELECT COUNT(*) FROM bank_transactions LIMIT 0, 1...	1 row(s) returned
70	15:07:11	SELECT amount FROM bank_transactions LIMIT 0, 1000	0 row(s) returned
71	15:07:11	INSERT INTO bank_transactions VALUES (101,'Ravi','H...	Error Code: 1136. Column count doesn't match value cou...
72	15:07:11	SELECT * FROM bank_transactions LIMIT 0, 1000	0 row(s) returned
73	15:07:54	USE bank_db	0 row(s) affected
74	15:08:29	CREATE DATABASE IF NOT EXISTS bank_db	1 row(s) affected, 1 warning(s): 1007 Can't create databas...
75	15:08:29	USE bank_db	0 row(s) affected
76	15:08:29	CREATE TABLE bank_transactions (txn_id INT PRIM...	Error Code: 1050. Table 'bank_transactions' already exists
77	15:08:29	DROP TABLE IF EXISTS bank_transactions	0 row(s) affected
78	15:08:43	CREATE DATABASE IF NOT EXISTS bank_db	1 row(s) affected, 1 warning(s): 1007 Can't create databas...
79	15:08:43	USE bank_db	0 row(s) affected
80	15:08:43	CREATE TABLE bank_transactions (txn_id INT PRIM...	0 row(s) affected
81	15:08:43	DROP TABLE IF EXISTS bank_transactions	0 row(s) affected
82	15:08:43	CREATE TABLE bank_transactions (txn_id INT PRIM...	0 row(s) affected
83	15:09:21	CREATE TABLE bank_transactions (txn_id INT PRIM...	Error Code: 1050. Table 'bank_transactions' already exists

Step 3: Insert Data

The screenshot shows the MySQL Workbench interface. The Navigator on the left displays the database schema, including tables like `bank_db`, `bank_manage`, `bookflow_db`, `emr_portal`, `joins_db`, `practice_db`, `sakila`, `student_db`, `student_info`, `student_mark`, and `students_db`. The SQL Editor in the center contains the following SQL code:

```
17 (105,'Priya','Hyderabad','Withdrawal',3500,'2024-01-11'),
18 (106,'Ramesh','Vizag','Deposit',15000,'2024-01-12'),
19 (107,'Keerthi','Vijayawada','Withdrawal',1000,'2024-01-13'),
20 (108,'Rahul','Hyderabad','Deposit',9000,'2024-01-14'),
21 (109,'Sneha','Vizag','Withdrawal',4000,'2024-01-15'),
22 (110,'Madhu','Vijayawada','Deposit',11000,'2024-01-16');
23 SELECT * FROM bank_transactions;
```

The Output pane at the bottom shows the execution results of the SQL statements. The first statement is an INSERT INTO statement, and the second is a SELECT * FROM statement. The output shows that 10 rows were affected by the INSERT statement and 10 rows were returned by the SELECT statement.

#	Time	Action	Message
84	15:10:02	INSERT INTO bank_transactions VALUES (101,'Ravi','H...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0
85	15:10:34	CREATE DATABASE IF NOT EXISTS bank_db	1 row(s) affected, 1 warning(s): 1007 Can't create databas...
86	15:10:34	USE bank_db	0 row(s) affected
87	15:10:34	DROP TABLE IF EXISTS bank_transactions	0 row(s) affected
88	15:10:34	CREATE TABLE bank_transactions (txn_id INT PRIM...	0 row(s) affected
89	15:10:34	INSERT INTO bank_transactions VALUES (101,'Ravi','H...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0
90	15:10:35	SELECT * FROM bank_transactions LIMIT 0, 1000	10 row(s) returned

Step 4: SUM — Total Amount

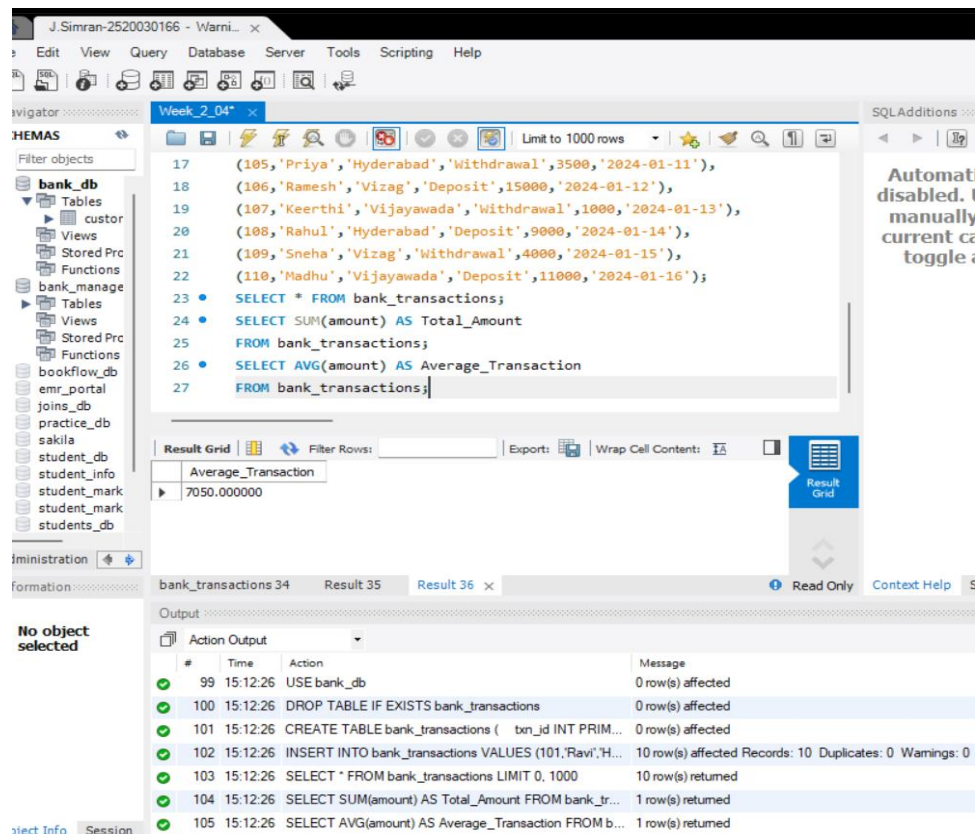
The screenshot shows the MySQL Workbench interface. The Navigator on the left displays the database schema, including tables like `bank_db`, `bank_manage`, `bookflow_db`, `emr_portal`, `joins_db`, `practice_db`, `sakila`, `student_db`, `student_info`, `student_mark`, and `students_db`. The SQL Editor in the center contains the following SQL code:

```
15 (103,'Kiran','Vijayawada','Deposit',12000,'2024-01-08'),
16 (104,'Anil','Vizag','Deposit',8000,'2024-01-10'),
17 (105,'Priya','Hyderabad','Withdrawal',3500,'2024-01-11'),
18 (106,'Ramesh','Vizag','Deposit',15000,'2024-01-12'),
19 (107,'Keerthi','Vijayawada','Withdrawal',1000,'2024-01-13'),
20 (108,'Rahul','Hyderabad','Deposit',9000,'2024-01-14'),
21 (109,'Sneha','Vizag','Withdrawal',4000,'2024-01-15'),
22 (110,'Madhu','Vijayawada','Deposit',11000,'2024-01-16');
23 SELECT * FROM bank_transactions;
24 SELECT SUM(amount) AS Total_Amount
25 FROM bank_transactions;
```

The Output pane at the bottom shows the execution results of the SQL statements. The first statement is an INSERT INTO statement, and the second is a SELECT SUM statement. The output shows that 10 rows were affected by the INSERT statement and 1 row was returned by the SELECT statement, with the total amount being 70500.00.

#	Time	Action	Message
91	15:11:20	CREATE DATABASE IF NOT EXISTS bank_db	1 row(s) affected, 1 warning(s): 1007 Can't create databas...
92	15:11:20	USE bank_db	0 row(s) affected
93	15:11:20	DROP TABLE IF EXISTS bank_transactions	0 row(s) affected
94	15:11:20	CREATE TABLE bank_transactions (txn_id INT PRIM...	0 row(s) affected
95	15:11:21	INSERT INTO bank_transactions VALUES (101,'Ravi','H...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0
96	15:11:21	SELECT * FROM bank_transactions LIMIT 0, 1000	10 row(s) returned
97	15:11:21	SELECT SUM(amount) AS Total_Amount FROM bank_tr...	1 row(s) returned

Step 5: AVG — Average Amount



The screenshot shows the SQL Developer interface with the following components:

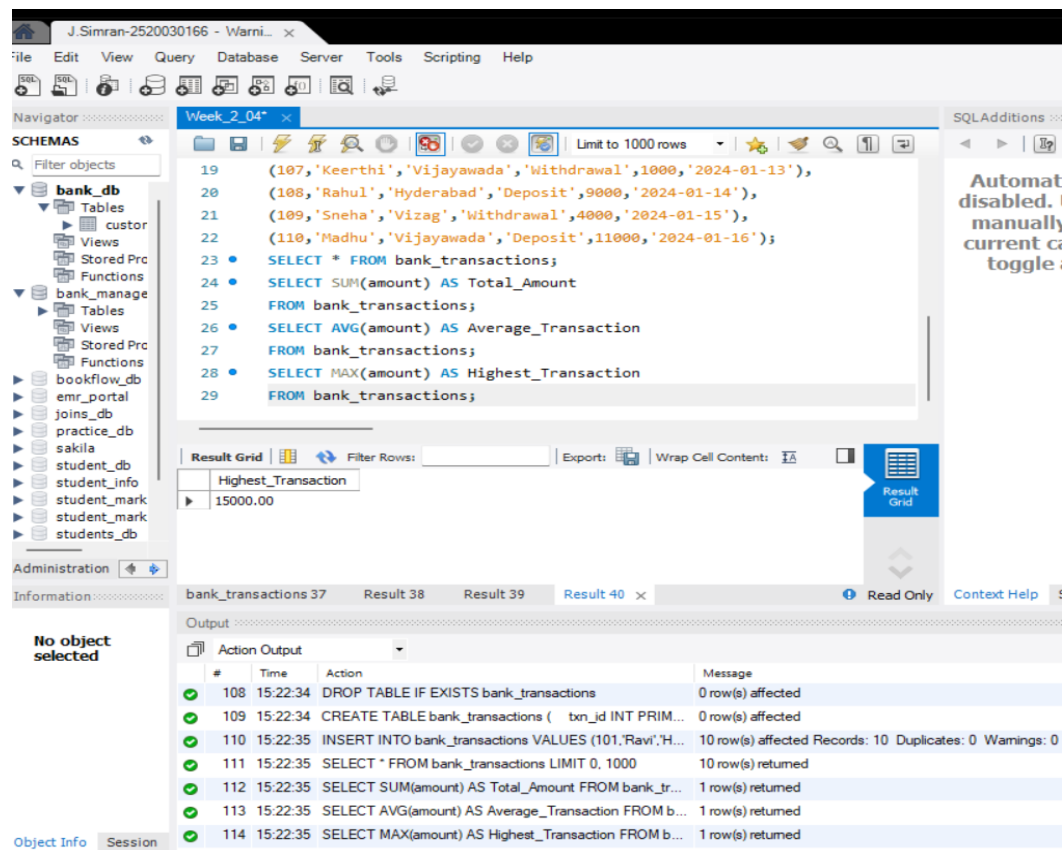
- Navigator:** Shows a tree view of database objects including `bank_db`, `bookflow_db`, `emr_portal`, `joins_db`, `practice_db`, `sakila`, `student_db`, `student_info`, `student_mark`, and `students_db`.
- Query Window:** Displays a SQL script with the following queries:

```
17 (105,'Priya','Hyderabad','Withdrawal',3500,'2024-01-11'),
18 (106,'Ramesh','Vizag','Deposit',15000,'2024-01-12'),
19 (107,'Keerthi','Vijayawada','Withdrawal',1000,'2024-01-13'),
20 (108,'Rahul','Hyderabad','Deposit',9000,'2024-01-14'),
21 (109,'Sneha','Vizag','Withdrawal',4000,'2024-01-15'),
22 (110,'Madhu','Vijayawada','Deposit',11000,'2024-01-16');
23 • SELECT * FROM bank_transactions;
24 • SELECT SUM(amount) AS Total_Amount
25 FROM bank_transactions;
26 • SELECT AVG(amount) AS Average_Transaction
27 FROM bank_transactions;
```
- Result Grid:** Shows the result of the `SELECT AVG` query:

Average_Transaction
7050.000000
- Output:** Shows the execution of the script with the following messages:

#	Time	Action	Message
99	15:12:26	USE bank_db	0 row(s) affected
100	15:12:26	DROP TABLE IF EXISTS bank_transactions	0 row(s) affected
101	15:12:26	CREATE TABLE bank_transactions (txn_id INT PRIM...	0 row(s) affected
102	15:12:26	INSERT INTO bank_transactions VALUES (101,'Ravi',H...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0
103	15:12:26	SELECT * FROM bank_transactions LIMIT 0, 1000	10 row(s) returned
104	15:12:26	SELECT SUM(amount) AS Total_Amount FROM bank_tr...	1 row(s) returned
105	15:12:26	SELECT AVG(amount) AS Average_Transaction FROM b...	1 row(s) returned

Step 6: MAX — Highest Amount



The screenshot shows the SQL Developer interface with the following components:

- Navigator:** Shows a tree view of database objects including `bank_db`, `bookflow_db`, `emr_portal`, `joins_db`, `practice_db`, `sakila`, `student_db`, `student_info`, `student_mark`, and `students_db`.
- Query Window:** Displays a SQL script with the following queries:

```
19 (107,'Keerthi','Vijayawada','Withdrawal',1000,'2024-01-13'),
20 (108,'Rahul','Hyderabad','Deposit',9000,'2024-01-14'),
21 (109,'Sneha','Vizag','Withdrawal',4000,'2024-01-15'),
22 (110,'Madhu','Vijayawada','Deposit',11000,'2024-01-16');
23 • SELECT * FROM bank_transactions;
24 • SELECT SUM(amount) AS Total_Amount
25 FROM bank_transactions;
26 • SELECT AVG(amount) AS Average_Transaction
27 FROM bank_transactions;
28 • SELECT MAX(amount) AS Highest_Transaction
29 FROM bank_transactions;
```
- Result Grid:** Shows the result of the `SELECT MAX` query:

Highest_Transaction
15000.00
- Output:** Shows the execution of the script with the following messages:

#	Time	Action	Message
108	15:22:34	DROP TABLE IF EXISTS bank_transactions	0 row(s) affected
109	15:22:34	CREATE TABLE bank_transactions (txn_id INT PRIM...	0 row(s) affected
110	15:22:35	INSERT INTO bank_transactions VALUES (101,'Ravi',H...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0
111	15:22:35	SELECT * FROM bank_transactions LIMIT 0, 1000	10 row(s) returned
112	15:22:35	SELECT SUM(amount) AS Total_Amount FROM bank_tr...	1 row(s) returned
113	15:22:35	SELECT AVG(amount) AS Average_Transaction FROM b...	1 row(s) returned
114	15:22:35	SELECT MAX(amount) AS Highest_Transaction FROM b...	1 row(s) returned

Step 7: MIN — Lowest Amount

The screenshot shows the SQL Developer interface with the following components:

- Navigator:** Shows a tree view of database schemas including `bank_db`, `bank_manage`, `bookflow_db`, `emr_portal`, `joins_db`, `practice_db`, `sakila`, `student_db`, `student_info`, `student_mark`, and `students_db`.
- Query Window:** Contains the following SQL code:

```
21 (109,'Sneha','Vizag','Withdrawal',4000,'2024-01-15'),
22 (110,'Madhu','Vijayawada','Deposit',11000,'2024-01-16');
23 • SELECT * FROM bank_transactions;
24 • SELECT SUM(amount) AS Total_Amount
25 FROM bank_transactions;
26 • SELECT AVG(amount) AS Average_Transaction
27 FROM bank_transactions;
28 • SELECT MAX(amount) AS Highest_Transaction
29 FROM bank_transactions;
30 • SELECT MIN(amount) AS Lowest_Transaction
31 FROM bank_transactions;
```
- Result Grid:** Displays the result for the `Lowest_Transaction` query, showing a single row with the value `1000.00`.
- Output Window:** Shows the execution log with the following messages:

#	Time	Action	Message
118	15:23:58	CREATE TABLE bank_transactions (txn_id INT PRIM...	0 row(s) affected
119	15:23:59	INSERT INTO bank_transactions VALUES (101,'Ravi','H...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0
120	15:23:59	SELECT * FROM bank_transactions LIMIT 0, 1000	10 row(s) returned
121	15:23:59	SELECT SUM(amount) AS Total_Amount FROM bank_tr...	1 row(s) returned
122	15:23:59	SELECT AVG(amount) AS Average_Transaction FROM b...	1 row(s) returned
123	15:23:59	SELECT MAX(amount) AS Highest_Transaction FROM b...	1 row(s) returned
124	15:23:59	SELECT MIN(amount) AS Lowest_Transaction FROM ba...	1 row(s) returned

Step 8: COUNT — Total Transactions

The screenshot shows the SQL Developer interface with the following components:

- Navigator:** Shows a tree view of database schemas including `bank_db`, `bank_manage`, `bookflow_db`, `emr_portal`, `joins_db`, `practice_db`, `sakila`, `student_db`, `student_info`, `student_mark`, and `students_db`.
- Query Window:** Contains the following SQL code:

```
23 • SELECT * FROM bank_transactions;
24 • SELECT SUM(amount) AS Total_Amount
25 FROM bank_transactions;
26 • SELECT AVG(amount) AS Average_Transaction
27 FROM bank_transactions;
28 • SELECT MAX(amount) AS Highest_Transaction
29 FROM bank_transactions;
30 • SELECT MIN(amount) AS Lowest_Transaction
31 FROM bank_transactions;
32 • SELECT COUNT(*) AS Total_Transactions
33 FROM bank_transactions;
```
- Result Grid:** Displays the result for the `Total_Transactions` query, showing a single row with the value `10`.
- Output Window:** Shows the execution log with the following messages:

#	Time	Action	Message
129	15:25:31	INSERT INTO bank_transactions VALUES (101,'Ravi','H...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0
130	15:25:31	SELECT * FROM bank_transactions LIMIT 0, 1000	10 row(s) returned
131	15:25:31	SELECT SUM(amount) AS Total_Amount FROM bank_tr...	1 row(s) returned
132	15:25:32	SELECT AVG(amount) AS Average_Transaction FROM b...	1 row(s) returned
133	15:25:32	SELECT MAX(amount) AS Highest_Transaction FROM b...	1 row(s) returned
134	15:25:32	SELECT MIN(amount) AS Lowest_Transaction FROM ba...	1 row(s) returned
135	15:25:32	SELECT COUNT(*) AS Total_Transactions FROM bank_t...	1 row(s) returned

Step 9: SUM + WHERE — Total Deposits

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the database structure, including 'bank_db' and its tables. The central pane contains a SQL query (lines 26-36) that calculates various statistics from the 'bank_transactions' table, filtered by 'transaction_type = 'Deposit''. The query includes: SELECT AVG(amount) AS Average_Transaction, FROM bank_transactions;, SELECT MAX(amount) AS Highest_Transaction, FROM bank_transactions;, SELECT MIN(amount) AS Lowest_Transaction, FROM bank_transactions;, SELECT COUNT(*) AS Total_Transactions, FROM bank_transactions;, SELECT SUM(amount) AS Total_Deposit, FROM bank_transactions, WHERE transaction_type = 'Deposit';. The right pane shows the 'Result Grid' with a single row: 'Total_Deposit' with a value of 60000.00. The bottom pane shows the 'Output' window with a log of execution messages, including 'SELECT SUM(amount) AS Total_Deposit FROM bank_transactions WHERE transaction_type = 'Deposit'' returning 1 row(s).

```
26 • SELECT AVG(amount) AS Average_Transaction
27 • FROM bank_transactions;
28 • SELECT MAX(amount) AS Highest_Transaction
29 • FROM bank_transactions;
30 • SELECT MIN(amount) AS Lowest_Transaction
31 • FROM bank_transactions;
32 • SELECT COUNT(*) AS Total_Transactions
33 • FROM bank_transactions;
34 • SELECT SUM(amount) AS Total_Deposit
35 • FROM bank_transactions
36 • WHERE transaction_type = 'Deposit';
```

Result Grid
Total_Deposit
60000.00

#	Time	Action	Message
✓ 141	15:26:26	SELECT * FROM bank_transactions LIMIT 0, 1000	10 row(s) returned
✓ 142	15:26:26	SELECT SUM(amount) AS Total_Deposit FROM bank_transactions WHERE transaction_type = 'Deposit'	1 row(s) returned
✓ 143	15:26:26	SELECT AVG(amount) AS Average_Transaction FROM bank_transactions	1 row(s) returned
✓ 144	15:26:26	SELECT MAX(amount) AS Highest_Transaction FROM bank_transactions	1 row(s) returned
✓ 145	15:26:26	SELECT MIN(amount) AS Lowest_Transaction FROM bank_transactions	1 row(s) returned
✓ 146	15:26:26	SELECT COUNT(*) AS Total_Transactions FROM bank_transactions	1 row(s) returned
✓ 147	15:26:26	SELECT SUM(amount) AS Total_Deposit FROM bank_transactions WHERE transaction_type = 'Deposit'	1 row(s) returned

Step 10: GROUP BY — Total by Branch

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the database structure. The central pane contains a SQL query (lines 30-40) that calculates statistics from the 'bank_transactions' table, filtered by 'transaction_type = 'Deposit'', and grouped by 'branch_name'. The query includes: SELECT MIN(amount) AS Lowest_Transaction, FROM bank_transactions;, SELECT COUNT(*) AS Total_Transactions, FROM bank_transactions;, SELECT SUM(amount) AS Total_Deposit, FROM bank_transactions, WHERE transaction_type = 'Deposit';, SELECT branch_name, SUM(amount) AS Total_Amount, FROM bank_transactions, GROUP BY branch_name;. The right pane shows the 'Result Grid' with three rows: 'Hyderabad' (19500.00), 'Vijayawada' (24000.00), and 'Vizag' (27000.00). The bottom pane shows the 'Output' window with a log of execution messages, including 'SELECT SUM(amount) AS Total_Amount FROM bank_transactions WHERE transaction_type = 'Deposit' GROUP BY branch_name' returning 3 row(s).

```
30 • SELECT MIN(amount) AS Lowest_Transaction
31 • FROM bank_transactions;
32 • SELECT COUNT(*) AS Total_Transactions
33 • FROM bank_transactions;
34 • SELECT SUM(amount) AS Total_Deposit
35 • FROM bank_transactions
36 • WHERE transaction_type = 'Deposit';
37 • SELECT branch_name,
38 • SUM(amount) AS Total_Amount
39 • FROM bank_transactions
40 • GROUP BY branch_name;
```

branch_name	Total_Amount
Hyderabad	19500.00
Vijayawada	24000.00
Vizag	27000.00

#	Time	Action	Message
✓ 154	15:28:22	SELECT SUM(amount) AS Total_Amount FROM bank_transactions WHERE transaction_type = 'Deposit' GROUP BY branch_name	1 row(s) returned
✓ 155	15:28:22	SELECT AVG(amount) AS Average_Transaction FROM bank_transactions	1 row(s) returned
✓ 156	15:28:22	SELECT MAX(amount) AS Highest_Transaction FROM bank_transactions	1 row(s) returned
✓ 157	15:28:22	SELECT MIN(amount) AS Lowest_Transaction FROM bank_transactions	1 row(s) returned
✓ 158	15:28:22	SELECT COUNT(*) AS Total_Transactions FROM bank_transactions	1 row(s) returned
✓ 159	15:28:22	SELECT SUM(amount) AS Total_Deposit FROM bank_transactions WHERE transaction_type = 'Deposit'	1 row(s) returned
✓ 160	15:28:22	SELECT branch_name, SUM(amount) AS Total_Amount FROM bank_transactions WHERE transaction_type = 'Deposit' GROUP BY branch_name	3 row(s) returned

Step 11: GROUP BY + HAVING

The screenshot shows the SQL Developer interface with a query window titled 'Week_2_04*'. The query is as follows:

```
35 FROM bank_transactions
36 WHERE transaction_type = 'Deposit';
37 SELECT branch_name,
38        SUM(amount) AS Total_Amount
39 FROM bank_transactions
40 GROUP BY branch_name;
41 SELECT branch_name,
42        SUM(amount) AS Total_Amount
43 FROM bank_transactions
44 GROUP BY branch_name
45 HAVING SUM(amount) > 20000;
```

The 'Result Grid' shows the following data:

branch_name	Total_Amount
Vijayawada	24000.00
Vizag	27000.00

The 'Output' pane shows the execution log with the following messages:

#	Time	Action	Message
168	15:29:09	SELECT AVG(amount) AS Average_Transaction FROM b...	1 row(s) returned
169	15:29:09	SELECT MAX(amount) AS Highest_Transaction FROM b...	1 row(s) returned
170	15:29:09	SELECT MIN(amount) AS Lowest_Transaction FROM ba...	1 row(s) returned
171	15:29:09	SELECT COUNT(*) AS Total_Transactions FROM bank_t...	1 row(s) returned
172	15:29:09	SELECT SUM(amount) AS Total_Deposit FROM bank_tr...	1 row(s) returned
173	15:29:09	SELECT branch_name, SUM(amount) AS Total_Amo...	3 row(s) returned
174	15:29:09	SELECT branch_name, SUM(amount) AS Total_Amo...	2 row(s) returned

Step 12: GROUP BY + ORDER BY

The screenshot shows the SQL Developer interface with a query window titled 'Week_2_04*'. The query is as follows:

```
40 GROUP BY branch_name;
41 SELECT branch_name,
42        SUM(amount) AS Total_Amount
43 FROM bank_transactions
44 GROUP BY branch_name
45 HAVING SUM(amount) > 20000;
46 SELECT branch_name,
47        SUM(amount) AS Total_Amount
48 FROM bank_transactions
49 GROUP BY branch_name
50 ORDER BY Total_Amount DESC;
```

The 'Result Grid' shows the following data:

branch_name	Total_Amount
Vizag	27000.00
Vijayawada	24000.00
Hyderabad	19500.00

The 'Output' pane shows the execution log with the following messages:

#	Time	Action	Message
183	15:29:44	SELECT MAX(amount) AS Highest_Transaction FROM b...	1 row(s) returned
184	15:29:44	SELECT MIN(amount) AS Lowest_Transaction FROM ba...	1 row(s) returned
185	15:29:44	SELECT COUNT(*) AS Total_Transactions FROM bank_t...	1 row(s) returned
186	15:29:44	SELECT SUM(amount) AS Total_Deposit FROM bank_tr...	1 row(s) returned
187	15:29:44	SELECT branch_name, SUM(amount) AS Total_Amo...	3 row(s) returned
188	15:29:44	SELECT branch_name, SUM(amount) AS Total_Amo...	2 row(s) returned
189	15:29:44	SELECT branch_name, SUM(amount) AS Total_Amo...	3 row(s) returned

Step 13: COUNT Withdrawals

The screenshot shows the SQL Developer interface with a query window titled "Week_2_04". The query is as follows:

```
43 FROM bank_transactions
44 GROUP BY branch_name
45 HAVING SUM(amount) > 20000;
46 SELECT branch_name,
47        SUM(amount) AS Total_Amount
48 FROM bank_transactions
49 GROUP BY branch_name
50 ORDER BY Total_Amount DESC;
51 SELECT COUNT(*) AS Withdrawals
52 FROM bank_transactions
53 WHERE transaction_type = 'Withdrawal';
```

The "Result Grid" shows a single row with the value 4 under the column "Withdrawals".

The "Output" pane shows a list of messages for the executed queries, including "SELECT COUNT(*) AS Total_Transactions FROM bank_transactions" and "SELECT COUNT(*) AS Withdrawals FROM bank_transactions".

Step 14: AVG Deposits

The screenshot shows the SQL Developer interface with a query window titled "Week_2_04". The query is as follows:

```
46 SELECT branch_name,
47        SUM(amount) AS Total_Amount
48 FROM bank_transactions
49 GROUP BY branch_name
50 ORDER BY Total_Amount DESC;
51 SELECT COUNT(*) AS Withdrawals
52 FROM bank_transactions
53 WHERE transaction_type = 'Withdrawal';
54 SELECT AVG(amount) AS Avg_Deposit
55 FROM bank_transactions
56 WHERE transaction_type = 'Deposit';
```

The "Result Grid" shows a single row with the value 10000.000000 under the column "Avg_Deposit".

The "Output" pane shows a list of messages for the executed queries, including "SELECT COUNT(*) AS Total_Transactions FROM bank_transactions", "SELECT COUNT(*) AS Withdrawals FROM bank_transactions", and "SELECT AVG(amount) AS Avg_Deposit FROM bank_transactions".

Step 15: MAX by Branch

J.Simran-2520030166 - Warni... x

Edit View Query Database Server Tools Scripting Help

Week_2_04*

Limit to 1000 rows

```

50 ORDER BY Total_Amount DESC;
51 • SELECT COUNT(*) AS Withdrawals
52 FROM bank_transactions
53 WHERE transaction_type = 'Withdrawal';
54 • SELECT AVG(amount) AS Avg_Deposit
55 FROM bank_transactions
56 WHERE transaction_type = 'Deposit';
57 • SELECT branch_name,
58     MAX(amount) AS Highest_Amount
59 FROM bank_transactions
60 GROUP BY branch_name;

```

branch_name	Highest_Amount
Hyderabad	9000.00
Vijayawada	12000.00
Vizag	15000.00

Result Grid

ministration

ormation

115 Result 116 Result 117 Result 118 Result 119 Result 120 Result 121 Read Only

Output

Action Output

#	Time	Action	Message
✓ 234	15:32:02	SELECT SUM(amount) AS Total_Deposit FROM bank_tr...	1 row(s) returned
✓ 235	15:32:02	SELECT branch_name, SUM(amount) AS Total_Amo...	3 row(s) returned
✓ 236	15:32:02	SELECT branch_name, SUM(amount) AS Total_Amo...	2 row(s) returned
✓ 237	15:32:02	SELECT branch_name, SUM(amount) AS Total_Amo...	3 row(s) returned
✓ 238	15:32:02	SELECT COUNT(*) AS Withdrawals FROM bank_transac...	1 row(s) returned
✓ 239	15:32:02	SELECT AVG(amount) AS Avg_Deposit FROM bank_tran...	1 row(s) returned
✓ 240	15:32:02	SELECT branch_name, MAX(amount) AS Highest_A...	3 row(s) returned

ject Info Session

Step 16: MIN by Branch

J.Simran-2520030166 - Warni... x

Edit View Query Database Server Tools Scripting Help

Week_2_04*

Limit to 1000 rows

```

54 • SELECT AVG(amount) AS Avg_Deposit
55 FROM bank_transactions
56 WHERE transaction_type = 'Deposit';
57 • SELECT branch_name,
58     MAX(amount) AS Highest_Amount
59 FROM bank_transactions
60 GROUP BY branch_name;
61 • SELECT branch_name,
62     MIN(amount) AS Lowest_Amount
63 FROM bank_transactions
64 GROUP BY branch_name;

```

branch_name	Lowest_Amount
Hyderabad	2000.00
Vijayawada	1000.00
Vizag	4000.00

Result Grid

ministration

ormation

129 Result 130 Result 131 Result 132 Result 133 Result 134 Result 135 Read Only

Output

Action Output

#	Time	Action	Message
✓ 253	15:32:56	SELECT branch_name, SUM(amount) AS Total_Amo...	3 row(s) returned
✓ 254	15:32:56	SELECT branch_name, SUM(amount) AS Total_Amo...	2 row(s) returned
✓ 255	15:32:56	SELECT branch_name, SUM(amount) AS Total_Amo...	3 row(s) returned
✓ 256	15:32:56	SELECT COUNT(*) AS Withdrawals FROM bank_transac...	1 row(s) returned
✓ 257	15:32:56	SELECT AVG(amount) AS Avg_Deposit FROM bank_tran...	1 row(s) returned
✓ 258	15:32:56	SELECT branch_name, MAX(amount) AS Highest_A...	3 row(s) returned
✓ 259	15:32:56	SELECT branch_name, MIN(amount) AS Lowest_Am...	3 row(s) returned

ject Info Session

Step 17: Transactions Greater Than 8000

The screenshot shows the SQL Developer interface with a query window titled 'Week_2_04*'. The query is as follows:

```
58      MAX(amount) AS Highest_Amount
59  FROM bank_transactions
60  GROUP BY branch_name;
61  • SELECT branch_name,
62      MIN(amount) AS Lowest_Amount
63  FROM bank_transactions
64  GROUP BY branch_name;
65  • SELECT *
66  FROM bank_transactions
67  WHERE amount > 8000
68  ORDER BY amount DESC;
```

The 'Result Grid' shows the results of the query:

txn_id	customer_name	branch_name	transaction_type	amount	transaction_date
106	Ramesh	Vizag	Deposit	15000.00	2024-01-12
103	Kiran	Vijayawada	Deposit	12000.00	2024-01-08
110	Madhu	Vijayawada	Deposit	11000.00	2024-01-16
108	Rahul	Hyderabad	Deposit	9000.00	2024-01-14

The 'Output' pane shows the execution log with the following messages:

#	Time	Action	Message
273	15:34:19	SELECT branch_name, SUM(amount) AS Total_Amo...	2 row(s) returned
274	15:34:19	SELECT branch_name, SUM(amount) AS Total_Amo...	3 row(s) returned
275	15:34:19	SELECT COUNT(*) AS Withdrawals FROM bank_transac...	1 row(s) returned
276	15:34:19	SELECT AVG(amount) AS Avg_Deposit FROM bank_tran...	1 row(s) returned
277	15:34:19	SELECT branch_name, MAX(amount) AS Highest_A...	3 row(s) returned
278	15:34:19	SELECT branch_name, MIN(amount) AS Lowest_Am...	3 row(s) returned
279	15:34:19	SELECT * FROM bank_transactions WHERE amount > 8...	4 row(s) returned

Step 18: Count Transactions by Branch

The screenshot shows the SQL Developer interface with a query window titled 'Week_2_04*'. The query is as follows:

```
62      MIN(amount) AS Lowest_Amount
63  FROM bank_transactions
64  GROUP BY branch_name;
65  • SELECT *
66  FROM bank_transactions
67  WHERE amount > 8000
68  ORDER BY amount DESC;
69  • SELECT branch_name,
70      COUNT(*) AS Customer_Count
71  FROM bank_transactions
72  GROUP BY branch_name;
```

The 'Result Grid' shows the results of the query:

branch_name	Customer_Count
Hyderabad	4
Vijayawada	3
Vizag	3

The 'Output' pane shows the execution log with the following messages:

#	Time	Action	Message
294	15:35:14	SELECT branch_name, SUM(amount) AS Total_Amo...	3 row(s) returned
295	15:35:14	SELECT COUNT(*) AS Withdrawals FROM bank_transac...	1 row(s) returned
296	15:35:14	SELECT AVG(amount) AS Avg_Deposit FROM bank_tran...	1 row(s) returned
297	15:35:14	SELECT branch_name, MAX(amount) AS Highest_A...	3 row(s) returned
298	15:35:14	SELECT branch_name, MIN(amount) AS Lowest_Am...	3 row(s) returned
299	15:35:14	SELECT * FROM bank_transactions WHERE amount > 8...	4 row(s) returned
300	15:35:14	SELECT branch_name, COUNT(*) AS Customer_Cou...	3 row(s) returned

Step 19: Branches With More Than 3 Transactions

The screenshot shows the SQL Developer interface with a query window titled "Week_2_04*". The query is as follows:

```
67 WHERE amount > 8000
68 ORDER BY amount DESC;
69 SELECT branch_name,
70        COUNT(*) AS Customer_Count
71 FROM bank_transactions
72 GROUP BY branch_name;
73 SELECT branch_name,
74        COUNT(*) AS Total_Transactions
75 FROM bank_transactions
76 GROUP BY branch_name
77 HAVING COUNT(*) > 3;
```

The "Result Grid" shows the following data:

branch_name	Total_Transactions
Hyderabad	4

The "Output" pane shows the following messages:

#	Time	Action	Message
316	15:36:05	SELECT COUNT(*) AS Withdrawals FROM bank_transac...	1 row(s) returned
317	15:36:05	SELECT AVG(amount) AS Avg_Deposit FROM bank_tran...	1 row(s) returned
318	15:36:05	SELECT branch_name, MAX(amount) AS Highest_A...	3 row(s) returned
319	15:36:05	SELECT branch_name, MIN(amount) AS Lowest_Am...	3 row(s) returned
320	15:36:05	SELECT * FROM bank_transactions WHERE amount > 8...	4 row(s) returned
321	15:36:05	SELECT branch_name, COUNT(*) AS Customer_Cou...	3 row(s) returned
322	15:36:05	SELECT branch_name, COUNT(*) AS Total_Transac...	1 row(s) returned

Step 20: Average by Branch

The screenshot shows the SQL Developer interface with a query window titled "Week_2_04*". The query is as follows:

```
71 FROM bank_transactions
72 GROUP BY branch_name;
73 SELECT branch_name,
74        COUNT(*) AS Total_Transactions
75 FROM bank_transactions
76 GROUP BY branch_name
77 HAVING COUNT(*) > 3;
78 SELECT branch_name,
79        AVG(amount) AS Avg_Amount
80 FROM bank_transactions
81 GROUP BY branch_name;
```

The "Result Grid" shows the following data:

branch_name	Avg_Amount
Hyderabad	4875.000000
Vijayawada	8000.000000
Vizag	9000.000000

The "Output" pane shows the following messages:

#	Time	Action	Message
339	15:36:41	SELECT AVG(amount) AS Avg_Deposit FROM bank_tran...	1 row(s) returned
340	15:36:41	SELECT branch_name, MAX(amount) AS Highest_A...	3 row(s) returned
341	15:36:42	SELECT branch_name, MIN(amount) AS Lowest_Am...	3 row(s) returned
342	15:36:42	SELECT * FROM bank_transactions WHERE amount > 8...	4 row(s) returned
343	15:36:42	SELECT branch_name, COUNT(*) AS Customer_Cou...	3 row(s) returned
344	15:36:42	SELECT branch_name, COUNT(*) AS Total_Transac...	1 row(s) returned
345	15:36:42	SELECT branch_name, AVG(amount) AS Avg_Amou...	3 row(s) returned

Step 21: Deposit Total Greater Than 15000

SQL Developer interface showing a query for Step 21. The query is as follows:

```
77 HAVING COUNT(*) > 3;
78 SELECT branch_name,
79        AVG(amount) AS Avg_Amount
80 FROM bank_transactions
81 GROUP BY branch_name;
82 SELECT branch_name,
83        SUM(amount) AS Total_Deposit
84 FROM bank_transactions
85 WHERE transaction_type = 'Deposit'
86 GROUP BY branch_name
87 HAVING SUM(amount) > 15000;
```

The result grid shows the following data:

branch_name	Total_Deposit
Vijayawada	23000.00
Vizag	23000.00

The output pane shows the execution of the query, with messages indicating the number of rows returned for each step.

Step 22: Average Greater Than 7000

SQL Developer interface showing a query for Step 22. The query is as follows:

```
83 SUM(amount) AS Total_Deposit
84 FROM bank_transactions
85 WHERE transaction_type = 'Deposit'
86 GROUP BY branch_name
87 HAVING SUM(amount) > 15000;
88 SELECT branch_name,
89        AVG(amount) AS Avg_Amount
90 FROM bank_transactions
91 GROUP BY branch_name
92 HAVING AVG(amount) > 7000
93 ORDER BY Avg_Amount DESC;
```

The result grid shows the following data:

branch_name	Avg_Amount
Vizag	9000.000000
Vijayawada	8000.000000

The output pane shows the execution of the query, with messages indicating the number of rows returned for each step.